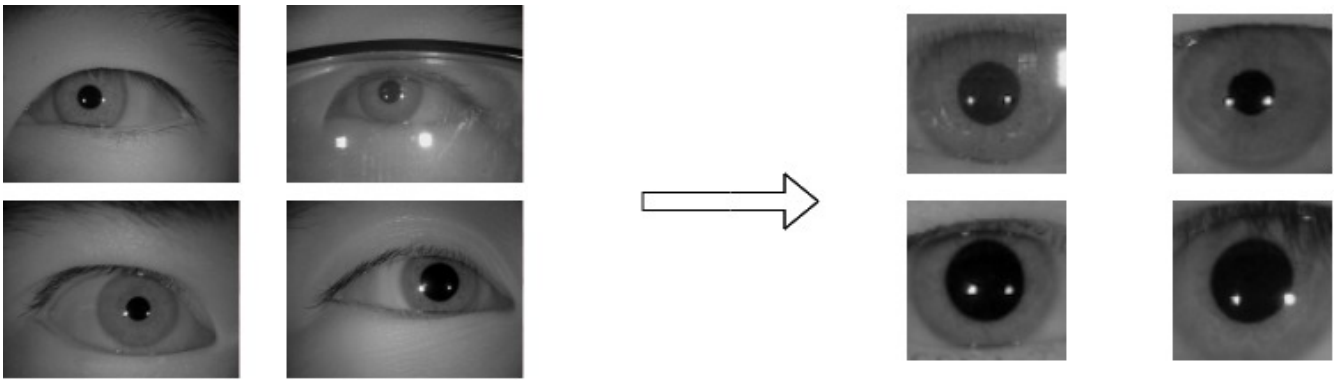
# Iris Vector Extraction System



## 📌 Project Overview

Welcome to my **Real-Time Iris Vector Extraction System**. This project is engineered to capture live video feeds from a webcam, dynamically detect the eye, perfectly segment the iris, and use a heavily modified Convolutional Neural Network (DenseNet-201) to output a 1920-dimensional feature vector in real-time.

This standalone system is completely optimized for live identity verification, embedding generation, and real-world deployment, shifting away from static dataset processing to dynamic, real-time AI processing.

---

## 🚀 Features

1. **Real-Time Live Feed Integration**: Process live video from standard Webcams or Infrared (IR) cameras effortlessly.
2. **Robust Eye & Iris Detection**: Uses Haar Cascades for initial eye region proposal and Hough Circle Transforms paired with mathematical morphology to accurately segment the iris.
3. **Deep Learning Feature Extraction**: Utilizes a "beheaded" DenseNet-201 architecture (stopping at `GlobalAveragePooling2D`) to generate rich 1920-dimension embeddings representing unique iris features.
4. **Self-Healing Pipeline**: Live feed execution is robust and skips frames without crashing if an eye or iris is momentarily undetectable.
5. **Hardware Optimization**: Includes utility scripts to export the underlying Keras/TensorFlow models to **Intel OpenVINO** format for extreme performance on CPU environments.

## How the System Works (The Full AI Pipeline)

This system relies on a multi-stage pipeline, blending classic computer vision (for fast localization) with modern deep learning (for highly accurate feature mapping).

### Stage 1: Eye Detection (`haarcascade_eye.xml`)

Before we can analyze an iris, we must locate the eye. We use a **Haar Cascade Classifier** (`haarcascade_eye.xml`), which is a classic, ultra-fast Machine Learning object detection model.

- **How it works**: It scans the image frame using a sliding window. At each step, it subtracts the sum of pixels under white rectangles from the sum of pixels under black rectangles (Haar-like features). It looks for specific contrast patterns—for example, the center of the eye (pupil/iris) is generally darker than the surrounding sclera (white part).
- **Why we use it**: It is extremely fast and lightweight, allowing us to drop background pixels instantly and focus computational power only on the bounding box containing the eye.

### Stage 2: Iris Segmentation (Mathematical Morphology & Hough Circles)

Once the eye is localized, the system must precisely isolate the iris (the colored ring) from the pupil (the dark center) and the sclera.

- **Image Processing**: The system converts the eye region to grayscale and applies Binary Thresholding combined with Mathematical Morphology (opening and closing operations). This removes eyelashes, specular reflections (light bouncing off the cornea), and skin noise.
- **Hough Circle Transform**: An algorithm that mathematically votes for the most likely circular shapes in the processed image. By setting specific radius constraints, it detects the exact center `(x, y)` and radius `(r)` of the iris, allowing the system to crop it perfectly.

### Stage 3: Deep Feature Extraction (DenseNet-201)

The cropped, perfectly aligned iris is passed to our deep learning model. We use **DenseNet-201** (Densely Connected Convolutional Network).

- **How DenseNet works**: Unlike traditional networks where a layer only connects to the next, DenseNet connects *every* layer to every subsequent layer. This allows it to learn incredibly complex, deep textures and micro-patterns inside the iris without losing information as the network gets deeper.
- **The "Beheaded" Architecture**: Normally, DenseNet outputs a specific class label (e.g., "Person A" or "Person B"). For our Real-Time service, we "behead" the model by removing its final classification layer and stopping at the `GlobalAveragePooling2D` layer.
- **The Output Vector**: Instead of a name, the network outputs a raw array of **1920 numbers (a high-dimensional embedding vector)**. This vector is a mathematical summary of the unique textures in the person's iris. In a real-world application, this vector can be compared against a database using Cosine Similarity or Euclidean

Distance to instantly verify identity.

---

## 📂 File Directory & Architecture

Here is a detailed breakdown of the core files inside the repository and their roles:

- **`real_time_iris.py`**: The core script for the live service. It opens your webcam (`cv2.VideoCapture`), detects the eye, extracts the iris ROI, and feeds it into the local AI model to print real-time embedding vectors to the console.
- **`iris_feature_extractor.h5`**: The pre-compiled, "beheaded" DenseNet-201 deep learning model. It is included locally in the directory so that no internet connection is required to download weights when delivering to the client.
- **`export_openvino.py`**: A utility script used to export the deep learning model. It saves the DenseNet-201 architecture as a TensorFlow SavedModel and provides the terminal commands required to convert it into OpenVINO IR format (`.xml` and `.bin`) for maximum Intel CPU efficiency.
- **`haarcascade_eye.xml`**: The pre-trained XML Haar Cascade classifier used by OpenCV to find eye regions instantly.

---

## 🛠️ Installation & Dependencies

To run this project, ensure you have Python installed along with the following libraries:

```
pip install numpy opencv-python tensorflow keras scikit-learn
```

If you plan to optimize the model for Intel hardware:

```
pip install openvino-dev
```

---

## 🏃♂️ How to Run the Project

You can run this project as a **Live Service** or perform **Model Optimization**.

### Method 1: Real-Time Iris Vector Extraction (Webcam)

This mode does not require downloading any dataset. It uses your attached camera.

1. Ensure your webcam is connected (An IR Camera is recommended for best results, but a standard RGB webcam works for testing).
2. Run the main service script:

   ```
   python real_time_iris.py
   ```

3. A window will pop up showing the camera feed. A blue rectangle will track your eye, and a green circle will track your iris.
4. The terminal will continuously output the 1920-dimension feature vector for your iris.
5. Press the `q` key on your keyboard to exit the live feed.

### Method 2: OpenVINO Model Optimization

To prepare the feature extractor for high-performance inference on Intel CPUs:

1. Run the export script:

   ```
   python export_openvino.py
   ```

2. The script will save the model to a `densenet201_beheaded` directory and print out the exact `mo` (Model Optimizer) command you need to run in your terminal to generate the `.xml` and `.bin` OpenVINO files.

---

# 🔍 Deep Dive: How the Core Files Work

This section provides a technical explanation of the two critical files that drive the AI pipeline: **haarcascade_eye.xml** and **iris_feature_extractor.h5**.

---

## 1. haarcascade_eye.xml (The Object Detector)

### What is it?

An XML file containing the structural definitions and mathematical weights of a pre-trained **Haar Cascade Classifier**. This classifier is specifically trained to recognize the spatial patterns of human eyes.

### How does it work?

- **Haar Features:** The file stores thousands of simple rectangular features (similar to edge, line, and four-rectangle features). For eyes, a common feature is that the region of the eye (pupil/iris) is generally darker than the surrounding area (bridge of the nose, cheeks).
- **Integral Images:** To compute these features rapidly in real-time video, OpenCV uses an "integral image" representation, allowing the difference between shaded and light regions to be calculated in constant time, regardless of the bounding box size.
- **Cascade of Classifiers:** The XML defines a "cascade" (a series of stages). When the script analyzes a frame, it passes sub-regions of the image through this cascade. The early stages contain very few simple features and reject the vast majority of non-eye background pixels instantly. Only regions that pass all stages are classified as an "eye".
- **Usage in code:** In `real_time_iris.py`, we use `cv2.CascadeClassifier('haarcascade_eye.xml')`. The `detectMultiScale` method slides a window across the image at different scales to find the bounding box where the cascade successfully triggers.

---

## 2. iris_feature_extractor.h5 (The Deep Learning Model)

### What is it?

An HDF5 (`.h5`) file that contains the entire architecture and the learned parameter weights (biases and filters) of a highly customized **DenseNet-201** neural network.

## How does it work?

- **Architecture Details:** The file encodes a Convolutional Neural Network (CNN) built with "Dense Blocks." Inside these blocks, every layer receives feature maps from all preceding layers. This dense connectivity encourages feature reuse and allows the network to learn the micro-textures of an iris (crypts, furrows, and freckles) without the vanishing gradient problem.
- **The "Beheaded" Structure:** A standard DenseNet-201 ends with a dense classification layer (e.g., softmax) to predict categories. This `.h5` file, however, has been "beheaded." Its final layer is a `GlobalAveragePooling2D` layer.
- **Feature Extraction:** Because the classification layer is removed, when an iris image is passed into the model (`self.model.predict(iris_input)`), the model outputs the raw activations of the pooling layer. This is a **1920-dimensional vector** (an embedding).
- **Why is this useful?** This 1920-dimensional mathematical vector acts as a digital fingerprint for the eye. By comparing the Euclidean distance or Cosine Similarity between two of these vectors, the system can determine if two eyes belong to the same person with extremely high precision.